Product Requirements Document

Predictive Pod Autoscaler
for Kubernetes

Project Name:	**Predictive Pod Autoscaler (PPA)**
Version:	1.0
Date:	February 2026
Status:	Planning Phase
Team:	Shubham Maity, Vatsal Umrania, Pratham Rathod
Department:	Information Technology
Timeline:	16 Weeks (Semester 6)

Table of Contents

- 1. Executive Summary
- 2. Problem Statement
- 3. Solution Overview
- 4. Functional Requirements
- 5. Non-Functional Requirements
- 6. System Architecture
- 7. Component Specifications
- 8. Data Requirements
- 9. Integration Requirements
- 10. User Stories & Use Cases
- 11. Success Metrics & KPIs
- 12. Development Timeline
- 13. Technical Stack
- 14. Testing Strategy
- 15. Risk Management
- 16. Deployment Strategy
- 17. Appendix

1. Executive Summary

1.1 Product Vision

The Predictive Pod Autoscaler (PPA) is an intelligent Kubernetes autoscaling solution that uses machine learning to predict application load 5-15 minutes in advance and proactively scales pods before traffic spikes occur. Unlike reactive solutions like Horizontal Pod Autoscaler (HPA), PPA prevents performance degradation by anticipating demand rather than responding to it.

1.2 Business Value

- Zero user-facing performance degradation during traffic spikes
- 20-30% reduction in infrastructure over-provisioning
- 50-70% faster scale-up response compared to HPA
- Improved application SLAs and user experience
- Reduced operational costs through efficient resource utilization

1.3 Target Users

- DevOps Engineers managing Kubernetes clusters
- Platform Teams responsible for infrastructure optimization
- SRE Teams maintaining production application uptime
- Development Teams deploying microservices on Kubernetes

2. Problem Statement

2.1 Current State Analysis

Modern cloud-native applications running on Kubernetes face a critical challenge: reactive autoscaling mechanisms (like HPA) only respond after load increases are detected, typically resulting in:

- Scale-up delay: 2-5 minutes from traffic spike to new pods being ready
- Performance degradation: Users experience slow response times or errors during scaling
- Resource waste: Teams over-provision to handle peak loads (30-40% average utilization)
- Manual intervention: On-call engineers must manually scale during known traffic events
- Unpredictable costs: Reactive scaling makes capacity planning difficult

2.2 Root Causes

Issue	Impact
HPA reacts only after CPU/memory thresholds are breached	Pods scale up after users already experience degradation
Polling interval (15-30 seconds) introduces lag	Detection delay compounds with pod startup time
No pattern recognition or learning	Cannot predict daily/weekly traffic patterns

2.3 Target Problem Scope

This project specifically addresses predictive autoscaling for stateless workloads based on request patterns, CPU, and memory metrics. Out of scope: database autoscaling, stateful sets, and multi-cluster orchestration.

3. Solution Overview

3.1 High-Level Approach

PPA employs a time-series machine learning model (LSTM or Prophet) trained on historical application metrics to forecast load 5-15 minutes ahead. A custom Kubernetes controller uses these predictions to preemptively scale deployments, ensuring capacity is ready before demand arrives.

3.2 Key Components

- Data Collection Layer: Prometheus + eBPF (Pixie) for enhanced metrics
- ML Prediction Engine: LSTM/Prophet model for load forecasting
- Kubernetes Controller: Custom operator that updates deployment replicas
- Monitoring Dashboard: Grafana for observability and performance tracking
- Validation Framework: Continuous feedback loop to improve predictions

3.3 Differentiation from Existing Solutions

Feature	HPA (Baseline)	KEDA	PPA (Ours)
Scaling Trigger	Reactive (after breach)	Event-based reactive	Predictive (before spike)
Lead Time	0 minutes	0 minutes	5-15 minutes
Learning Capability	None	None	ML-based pattern learning
Scale-Up Speed	2-5 minutes	1-3 minutes	<1 minute (pre-scaled)

4. Functional Requirements

4.1 Data Collection (FR-DC)

- **FR-DC-01:** System MUST collect CPU and memory metrics from Prometheus at 15-second intervals
- **FR-DC-02:** System MUST collect HTTP request rate, latency (P95, P99), and error rate via eBPF
- **FR-DC-03:** System MUST store at least 30 days of historical metrics for model training
- **FR-DC-04:** System MUST handle metric collection failures gracefully without crashing
- **FR-DC-05:** System SHOULD collect active connection count and network throughput (optional)

4.2 Machine Learning Prediction (FR-ML)

- **FR-ML-01:** Model MUST predict application load 5, 10, and 15 minutes into the future
- **FR-ML-02:** Model MUST achieve <15% MAPE (Mean Absolute Percentage Error) on test data
- **FR-ML-03:** Model MUST retrain automatically every 7 days using recent data
- **FR-ML-04:** Model inference MUST complete within 100ms to enable real-time scaling
- **FR-ML-05:** System MUST detect concept drift and trigger retraining when accuracy degrades >20%
- **FR-ML-06:** Model SHOULD support both LSTM and Prophet algorithms for comparison

4.3 Kubernetes Scaling Controller (FR-K8S)

- **FR-K8S-01:** Controller MUST update Deployment replica count based on ML predictions
- **FR-K8S-02:** Controller MUST enforce minimum replica count (default: 1) and maximum (default: 20)
- **FR-K8S-03:** Controller MUST implement rate limiting: max 2x scale-up per 2-minute window
- **FR-K8S-04:** Controller MUST log all scaling decisions with timestamp, prediction, and actual load
- **FR-K8S-05:** Controller MUST provide rollback capability if prediction accuracy drops below threshold
- **FR-K8S-06:** Controller SHOULD support dry-run mode for testing without actual scaling

4.4 Monitoring and Observability (FR-MON)

- **FR-MON-01:** System MUST expose prediction accuracy metrics to Prometheus
- **FR-MON-02:** System MUST provide Grafana dashboard showing predictions vs actual load
- **FR-MON-03:** System MUST track scaling events: timestamp, trigger reason, outcome
- **FR-MON-04:** System MUST send alerts when prediction error exceeds 25% for >5 minutes
- **FR-MON-05:** System SHOULD provide explainability: why was scaling decision made?

5. Non-Functional Requirements

5.1 Performance (NFR-PERF)

- **NFR-PERF-01:** ML prediction latency: <100ms (P95)
- **NFR-PERF-02:** Controller reconciliation loop: <30 seconds
- **NFR-PERF-03:** Metrics collection overhead: <5% CPU on monitored pods
- **NFR-PERF-04:** System can handle clusters with up to 100 deployments

5.2 Reliability (NFR-REL)

- **NFR-REL-01:** Controller uptime: 99.5% (allows 3.6 hours downtime/month)
- **NFR-REL-02:** Graceful degradation: Fall back to HPA if ML model fails
- **NFR-REL-03:** Zero data loss: All predictions and metrics persisted
- **NFR-REL-04:** Automated recovery: Controller restarts on crash within 30 seconds

5.3 Scalability (NFR-SCALE)

- **NFR-SCALE-01:** Support 1-100 deployments per cluster
- **NFR-SCALE-02:** Handle 10,000+ metrics data points per deployment
- **NFR-SCALE-03:** ML model storage: <500MB per trained model
- **NFR-SCALE-04:** Controller memory footprint: <2GB at 100 deployments

5.4 Security (NFR-SEC)

- **NFR-SEC-01:** Controller uses Kubernetes RBAC with least-privilege service account
- **NFR-SEC-02:** TLS encryption for all external API calls
- **NFR-SEC-03:** No sensitive data (credentials, tokens) in logs
- **NFR-SEC-04:** Model artifacts signed and versioned for integrity

5.5 Usability (NFR-USE)

- **NFR-USE-01:** Installation via single Helm chart command
- **NFR-USE-02:** Configuration through Kubernetes CustomResource (CRD)
- **NFR-USE-03:** Documentation includes setup guide, troubleshooting, and examples
- **NFR-USE-04:** Grafana dashboard pre-configured and ready-to-use

6. System Architecture

6.1 Architecture Diagram

[Architecture diagram showing: Kubernetes Cluster → eBPF Agent → Metrics Storage → ML Engine → K8s Controller → Deployment Updates]

(Diagram to be created using draw.io or similar tool)

6.2 Component Interaction Flow

The system operates in a continuous feedback loop:

1. 1. eBPF Agent (Pixie) and Prometheus collect real-time metrics from pods
2. 2. Metrics are stored in Prometheus time-series database
3. 3. ML Prediction Engine queries historical data every 30 seconds
4. 4. Model generates load forecast for next 5, 10, 15 minutes
5. 5. Kubernetes Controller evaluates predictions against current state
6. 6. If scaling needed, controller updates Deployment replica count
7. 7. Kubernetes schedules new pods (or terminates excess)
8. 8. Actual load vs prediction is logged for model retraining
9. 9. Grafana dashboard visualizes predictions, actuals, and scaling events

6.3 Data Flow

Metrics → Prometheus → Feature Engineering → ML Model → Prediction → Controller → Kubernetes API → Pod Scaling → Validation → Model Feedback

7. Component Specifications

7.1 Data Collection Layer

Prometheus Configuration

Metrics to collect:

- `container_cpu_usage_seconds_total`
- `container_memory_usage_bytes`
- `http_requests_total (application-level)`
- `http_request_duration_seconds`

eBPF Agent (Pixie)

Enhanced metrics:

- HTTP latency histogram (P50, P95, P99)
- Active connection count
- Request error rate (4xx, 5xx)
- Network throughput (bytes in/out)
- Per-endpoint performance breakdown

7.2 ML Prediction Engine

Model Architecture

Primary Model: LSTM (Long Short-Term Memory)

- Input: 60-minute window of metrics (5-minute intervals = 12 data points)
- Hidden layers: 2 LSTM layers (64 units, 32 units)
- Output: Predicted load at t+5, t+10, t+15 minutes
- Training: 30 days historical data, 80/20 train/validation split
- Loss function: Mean Squared Error (MSE)
- Optimizer: Adam with learning rate 0.001

Feature Engineering

Input features per time step:

- requests_per_second (from HTTP metrics)
- cpu_usage_percent (from cAdvisor)
- memory_usage_bytes (from cAdvisor)
- latency_p95_ms (from eBPF)
- active_connections (from eBPF)
- error_rate (from eBPF)
- hour_of_day (temporal feature)
- day_of_week (temporal feature)

- is_weekend (boolean feature)

7.3 Kubernetes Controller

Custom Resource Definition (CRD)

Example configuration:

```
apiVersion: ppa.example.com/v1
kind: PredictiveAutoscaler
metadata:
  name: my-app-autoscaler
spec:
  targetDeployment: my-app
  minReplicas: 2
  maxReplicas: 20
  predictionHorizon: 10m
  modelType: lstm
  scalingPolicy:
    scaleUpRate: 2  # Max 2x increase per cycle
    scaleDownRate: 0.5  # Max 50% decrease per cycle
    stabilizationWindow: 2m
```

Controller Logic

10. 1. Reconciliation loop runs every 30 seconds
11. 2. Query ML engine for load prediction
12. 3. Calculate required replicas: ceil(predicted_load / capacity_per_pod)
13. 4. Apply safety constraints (min/max replicas, rate limits)
14. 5. If change needed, update Deployment.spec.replicas
15. 6. Log decision with rationale and timestamp
16. 7. Emit metrics for monitoring

8. Data Requirements

8.1 Training Data

- Minimum dataset: 30 days of production metrics
- Preferred dataset: 90 days for seasonal pattern detection
- Data frequency: 1-minute granularity
- Quality requirements: <10% missing data points
- Diversity: Include normal operations, traffic spikes, and low-traffic periods

8.2 Storage Requirements

Data Type	Retention Period	Estimated Size
Prometheus metrics	30 days	~10 GB per deployment
ML model checkpoints	90 days (last 3 versions)	~500 MB per model
Prediction logs	30 days	~1 GB per deployment

8.3 Data Privacy & Compliance

- No collection of user PII (personally identifiable information)
- Metrics are aggregated at pod/deployment level only
- HTTP payload data is not captured (only metadata like status codes)
- All data stored within cluster boundaries (no external transmission)

9. Integration Requirements

9.1 Kubernetes Integration

- Kubernetes version: 1.25+ (API compatibility)
- Required APIs: apps/v1 (Deployments), metrics.k8s.io (Metrics Server)
- RBAC permissions: get, list, watch, update on Deployments
- CRD registration: ppa.example.com/v1alpha1
- Admission webhooks: Optional (for validation)

9.2 Monitoring Stack Integration

- Prometheus: Scrape interval 15s, remote write optional
- Grafana: Version 9.0+, pre-configured dashboard JSON
- Pixie (eBPF): px CLI for installation, API for querying
- AlertManager: Optional, for prediction accuracy alerts

9.3 CI/CD Integration

- GitHub Actions for automated testing and building
- Docker Hub or private registry for container images
- Helm chart repository for distribution
- ArgoCD/FluxCD support for GitOps deployment (future)

10. User Stories & Use Cases

10.1 User Stories

US-01

Actor: DevOps Engineer

Story: As a DevOps engineer, I want the system to automatically scale my application before traffic spikes, so that users never experience degraded performance.

Acceptance Criteria: *Pods are scaled up at least 2 minutes before traffic spike hits, zero 5xx errors during scaling event.*

US-02

Actor: SRE Team Lead

Story: As an SRE team lead, I want to monitor prediction accuracy in real-time, so that I can trust the autoscaler and debug issues quickly.

Acceptance Criteria: *Grafana dashboard shows predicted vs actual load with <15% MAPE, alerts trigger if accuracy degrades.*

US-03

Actor: Platform Engineer

Story: As a platform engineer, I want to deploy the autoscaler with minimal configuration, so that teams can adopt it without extensive training.

Acceptance Criteria: *Single Helm command deploys system, CRD configuration requires only 5 fields (target deployment, min/max replicas, model type).*

US-04

Actor: Cost Optimization Manager

Story: As a cost manager, I want to reduce over-provisioning waste, so that we save on cloud infrastructure costs.

Acceptance Criteria: *Average pod utilization increases from 40% to 65%, monthly cloud bill reduced by 20%.*

10.2 Primary Use Case Flow

UC-01: Predictive Scaling During Morning Traffic Spike

17. Preconditions: System deployed, ML model trained, application running with 5 pods
18. Trigger: 8:45 AM - ML model predicts traffic will spike at 9:00 AM
19. Step 1: Model forecasts load increase from 100 req/s to 500 req/s in 15 minutes
20. Step 2: Controller calculates required replicas: 25 pods (based on 20 req/s per pod)
21. Step 3: Controller begins scaling: 5 → 10 → 15 → 20 → 25 pods over 10 minutes
22. Step 4: By 8:55 AM, all 25 pods are ready and healthy
23. Step 5: At 9:00 AM, traffic spike arrives - application handles load smoothly
24. Step 6: Zero performance degradation, P95 latency stays below 200ms
25. Postconditions: Application maintained SLA, prediction logged for model improvement

11. Success Metrics & KPIs

11.1 Primary Success Metrics

Metric	Baseline (HPA)	Target (PPA)	Measurement Method
Prediction Accuracy (MAPE)	N/A	<15%	Compare predicted vs actual load
Scale-Up Latency	2-5 minutes	<1 minute	Time from prediction to pods ready
Performance Degradation Events	5-10 per month	0 per month	Count of P95 latency >500ms
Over-Provisioning Rate	35%	<20%	% of time pods are <50% utilized
Resource Cost Savings	Baseline	20% reduction	Monthly cloud bill comparison
System Availability	99.0%	99.5%	Controller uptime tracking

11.2 Secondary Metrics

- False Positive Rate: <10% (unnecessary scaling events)
- False Negative Rate: <5% (missed scaling opportunities)
- Model Inference Latency: <100ms (P95)
- Controller Reconciliation Time: <30 seconds
- Time to Recovery from Controller Failure: <60 seconds

11.3 Evaluation Plan

- Week 11-13: Deploy PPA alongside HPA in staging environment
- Week 13: Generate synthetic traffic patterns (morning spike, flash sale, gradual growth)
- Week 14: Run A/B test: HPA for 7 days, then PPA for 7 days on same workload
- Week 15: Analyze results, generate comparison report
- Deliverable: Performance comparison document with graphs and statistical analysis

12. Development Timeline

Phase	Weeks	Deliverables	Owner
Setup & Planning	1-3	Cluster setup, Prometheus, Pixie deployed, 30 days data collected	All
ML Model Development	4-7	LSTM model trained, <15% MAPE achieved, inference API ready	Shubham
K8s Controller	8-10	Custom operator coded, CRD defined, basic scaling working	Vatsal
Integration	11-13	ML + Controller integrated, end-to-end flow tested	Pratham
Benchmarking	14-15	HPA comparison complete, performance report generated	All
Documentation	16	README, API docs, setup guide, demo video	All
Buffer	16	Bug fixes, final polish, presentation prep	All

12.1 Milestones

- M1 (End Week 3): Data collection pipeline operational
- M2 (End Week 7): ML model achieves <15% MAPE on validation set
- M3 (End Week 10): Controller successfully scales test deployment
- M4 (End Week 13): Full system deployed in staging environment
- M5 (End Week 15): Benchmark results demonstrate 50%+ improvement over HPA
- M6 (End Week 16): All deliverables complete, ready for presentation

13. Technical Stack

13.1 Infrastructure

- Kubernetes: v1.25+ (Minikube or Kind for local, GKE/EKS for cloud)
- Container Runtime: Docker 20.10+
- Networking: Calico or default CNI
- Storage: Local volumes for dev, CSI driver for production

13.2 Monitoring & Observability

- Prometheus: 2.40+ (time-series metrics storage)
- Grafana: 9.0+ (visualization and dashboards)
- Pixie: Latest stable (eBPF-based metrics)
- Metrics Server: v0.6+ (K8s resource metrics)

13.3 Machine Learning

- Language: Python 3.9+
- Framework: TensorFlow 2.11+ or PyTorch 2.0+
- Model: LSTM (primary), Prophet (baseline comparison)
- Libraries: NumPy, Pandas, Scikit-learn, Matplotlib
- Training: Jupyter notebooks for experimentation
- Serving: Flask API for model inference

13.4 Kubernetes Controller

- Language: Python 3.9+ or Go 1.20+
- Framework: Kopf (Python Kubernetes operator framework)
- Alternative: Kubebuilder (if using Go)
- K8s Client: kubernetes-client/python or client-go
- Logging: Python logging or logrus (Go)

13.5 Development Tools

- Version Control: Git + GitHub
- CI/CD: GitHub Actions
- Containerization: Docker
- Package Management: Helm 3.10+
- Testing: pytest (Python), k3s (lightweight K8s for testing)
- Load Testing: Locust or K6

14. Testing Strategy

14.1 Unit Testing

- ML Model: Test feature engineering, model inference, prediction output format
- Controller: Test replica calculation logic, rate limiting, min/max constraints
- Data Collection: Test metric parsing, error handling, data validation
- Coverage Target: >80% code coverage
- Tools: pytest for Python, go test for Go

14.2 Integration Testing

- Prometheus Integration: Verify metrics are queryable and correctly formatted
- Pixie Integration: Ensure eBPF data flows to ML pipeline
- K8s API Integration: Test deployment updates, RBAC permissions
- End-to-end Flow: Metrics → Prediction → Scaling → Validation
- Environment: Minikube cluster with test deployment

14.3 Performance Testing

- Load Testing: Generate traffic patterns with Locust (gradual, spike, oscillating)
- Latency Testing: Measure prediction and scaling response times
- Stress Testing: Test system with 100 concurrent deployments
- Baseline Comparison: Run identical workload with HPA vs PPA

14.4 Scenario Testing

Test Cases:

- TC-01: Morning traffic spike (gradual increase over 30 minutes)
- TC-02: Flash sale event (sudden 10x spike in 1 minute)
- TC-03: Gradual evening decline (scale-down testing)
- TC-04: Weekend low traffic (minimum replica enforcement)
- TC-05: Model failure (fallback to HPA or conservative scaling)
- TC-06: Controller restart (state recovery)
- TC-07: Network partition (graceful degradation)

15. Risk Management

Risk	Probability	Impact	Mitigation Strategy
ML model accuracy below 15% MAPE	Medium	High	Start with Prophet baseline, add more features, extend training data to 60 days
Kubernetes API changes break controller	Low	High	Pin K8s API version, extensive integration tests, graceful error handling
Insufficient training data	Medium	Medium	Use synthetic traffic generators (Locust) to create realistic patterns
Team skill gaps in K8s operators	High	Medium	Week 1-2: Dedicated learning time, pair programming, use Kopf framework
Scope creep (adding extra features)	High	High	Strict adherence to PRD, weekly scope reviews, defer features to "Future Work"
Controller causes cluster instability	Low	Critical	Dry-run mode, rate limiting, max replica caps, extensive staging tests before production

16. Deployment Strategy

16.1 Deployment Phases

- Phase 1: Local Development (Weeks 1-10)
- - Minikube or Kind cluster on developer machines
- - Individual component testing
- Phase 2: Staging Environment (Weeks 11-13)
- - Cloud-based cluster (GKE/EKS)
- - Full system integration
- - Synthetic load testing
- Phase 3: Production Dry-Run (Week 14)
- - Deploy PPA in monitoring-only mode
- - Log scaling decisions without actually scaling
- - Verify predictions against real traffic
- Phase 4: Controlled Rollout (Week 15)
- - Enable active scaling on non-critical deployment
- - Monitor closely for 48 hours
- - Expand to additional deployments if successful

16.2 Rollback Plan

- Trigger Conditions: Prediction accuracy <50%, controller crashes >3 times, user-reported performance issues
- Immediate Action: Disable PPA controller, fall back to HPA
- Root Cause Analysis: Review logs, metrics, model performance
- Fix & Redeploy: Address issues in staging before re-enabling
- Rollback Time Objective: <5 minutes from decision to HPA active

16.3 Installation Guide

Simplified installation steps:

```
# 1. Install prerequisites
kubectl apply -f https://github.com/prometheus-operator/prometheus-
operator/releases/latest/download/bundle.yaml
px deploy  # Install Pixie

# 2. Install PPA via Helm
helm repo add ppa https://github.com/your-team/ppa-charts
helm install my-ppa ppa/predictive-pod-autoscaler \
  --set model.type=lstm \
  --set monitoring.grafana.enabled=true

# 3. Create autoscaler for your deployment
kubectl apply -f - <<EOF
apiVersion: ppa.example.com/v1
kind: PredictiveAutoscaler
metadata:
  name: my-app-autoscaler
spec:
```

```
  targetDeployment: my-app
  minReplicas: 2
  maxReplicas: 20
  predictionHorizon: 10m
EOF

# 4. Verify installation
kubectl get predictiveautoscalers
kubectl logs -l app=ppa-controller
```

17. Appendix

17.1 Glossary

Term	Definition
HPA	Horizontal Pod Autoscaler - Kubernetes built-in reactive autoscaling
LSTM	Long Short-Term Memory - Type of recurrent neural network for time-series
MAPE	Mean Absolute Percentage Error - Prediction accuracy metric
eBPF	Extended Berkeley Packet Filter - Kernel-level observability technology
CRD	Custom Resource Definition - Kubernetes extension mechanism
Prophet	Facebook's time-series forecasting library
P95/P99	95th/99th percentile - Statistical measure of latency
Kopf	Kubernetes Operator Pythonic Framework
Pod	Smallest deployable unit in Kubernetes (one or more containers)

17.2 References

26. [1] AutoPilot AI: Architecting Self-Healing ML Systems with Reinforcement Feedback Loops (Nellutla et al., 2025)
27. [2] Self-healing Application Architecture for Cloud-Based Environments (Gattupalli et al., 2025)
28. [3] Kubernetes Documentation: Horizontal Pod Autoscaler (https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/)
29. [4] Prometheus Documentation (https://prometheus.io/docs/)
30. [5] Pixie Documentation (https://docs.px.dev/)
31. [6] Facebook Prophet (https://facebook.github.io/prophet/)
32. [7] Kopf Framework (https://kopf.readthedocs.io/)
33. [8] LSTM Networks for Time Series Forecasting (Hochreiter & Schmidhuber, 1997)

17.3 Acronyms

- API - Application Programming Interface
- CPU - Central Processing Unit
- CRD - Custom Resource Definition
- DXA - Twentieth of a Point (Word measurement unit)
- eBPF - Extended Berkeley Packet Filter
- HPA - Horizontal Pod Autoscaler
- K8s - Kubernetes (8 letters between K and s)

- KPI - Key Performance Indicator
- LSTM - Long Short-Term Memory
- MAPE - Mean Absolute Percentage Error
- ML - Machine Learning
- MSE - Mean Squared Error
- NFR - Non-Functional Requirement
- PPA - Predictive Pod Autoscaler
- RBAC - Role-Based Access Control
- SLA - Service Level Agreement
- SRE - Site Reliability Engineering
- TLS - Transport Layer Security

17.4 Contact Information

Name	Role	Responsibilities
Shubham Maity	ML Engineer	Model development, training, evaluation
Vatsal Umrania	DevOps Engineer	K8s controller, infrastructure, deployment
Pratham Rathod	Integration Lead	System integration, testing, documentation